Fail-Closed Governed Dataset Lifecycles and Explicit Training Authorization for AI Training Systems

Ed25519 decision authentication, complete package sealing and evidence-bound execution

Document identifier: APOD-TR-010.

Author: Anthony Siljeg.
Affiliation: Alnotrea Pty Ltd / Policy of Developments - Access-PoD Initiative.
Edition: Q-Pathformer Enterprise v2.0 Standards Edition.
Document date: 6 September 2026.
Status: Final manuscript - v2.0 Standards Edition.
Companion: APOD-TR-010-R, Verification Results and v1.0 Comparison, compact standards edition.
Contribution: An implementation report and a reusable reference protocol for evidence-bound transitions.
Review boundary: No actual lawyer review or independent assurance is claimed.

Abstract

A dataset can be technically ready without being authorized for use. Similarly, a valid signature can identify a signing key without establishing the signer's legal authority, and an intact package can still contain unsuitable material. Treating these different findings as interchangeable can allow a system to proceed beyond its recorded permission.

APOD-TR-010 describes how the Q-Pathformer Enterprise reference implementation separates lifecycle eligibility, a recorded human decision, package integrity and execution-time checks. It has a dual role: an implementation report and a reusable evidence-bound transition protocol. The protocol explains what an identified transition consumes, which conditions are checked, what decision is recorded and which outputs may follow. APOD-TR-010-R provides the companion verification results and their limits.

The current standards evidence is a bounded September 2026 synthetic exercise, not a new enterprise training run. A retained suite lists 48 tests: 47 passed, one was skipped for Windows symbolic-link capability, and none failed. A separate nine-step command-line rehearsal matched all expected outcomes, including intentional rejections. It begins with four synthetic records already represented in a STATE_1, freeze and registry fixture. It does not reproduce the complete three-path lifecycle, train a model or produce a new CSC score. Earlier v1.0 execution and July v2 candidate records remain separately identified historical evidence.

The proposed project profile reserves assessment of identified legal questions to qualified human review and operational permission to the responsible organization. The implementation does not verify practitioner credentials or automatically approve legal rights. Its contribution is narrower: making specified technical conditions and recorded authorization independently inspectable without treating either as a substitute for the other human responsibilities. This paper forms part of the compact v2.0 Standards Edition. The v3 enterprise release remains planned.

Keywords: Governed datasets; explicit authorization; package integrity; Ed25519; provenance; standards exploration.

Edition scope and history

Q-Pathformer Enterprise v2.0 Standards Edition is the current compact standards-review edition. APOD-TR-010 explains the implemented control method and reusable evidence-bound transition protocol. APOD-TR-010-R reports the bounded September synthetic verification and separately identified historical comparisons. The v2 edition does not reproduce the complete v1.0 enterprise lifecycle. The planned v3 enterprise release is intended to combine the v1.0 three-path foundation with the v2 authorization and package-integrity controls for end-to-end reproduction within a defined enterprise scope. V3 is not released, and its intended outcomes are not verified v2 results. Future CSC Assurance Record work remains separately scoped; it is not made operational by publishing these papers.

The companion `history_notice.md` records the longer chronology and provenance boundary. TR010-R identifies the retained evidence and its limitations. Original manuscripts and historical ledgers retain their dated identities. Earlier editorial handoffs describe preceding manuscript snapshots; the final publication PDFs are identified separately after conversion and review.

Contents

1 Problem and foundations 6
1.1 Readiness is not permission 6
1.2 The two contributions 7
1.3 Relationship to the earlier papers 7
1.4 Four classes of statement 8
1.5 Evidence and release identities 9
2 Lifecycle control model 10
2.1 Selection is an input decision 10
2.2 Missing evidence remains missing 11
2.3 Technical review and human decision 11
2.4 Transition invariants 11
2.5 Positive outputs and held outputs 12
2.6 Freeze and registry are not operation grants 13
2.7 What the demonstration omits 13
3 Authorization contract 14
3.1 Decision and scope 14
3.2 Identity and time 14
3.3 What each endpoint actually consumes 15
3.4 Recorded policy versus enforced policy 16
3.5 Structure before interpretation 16
3.6 Decision creation and acknowledgment 17
3.7 Authentic but unacceptable cases 17
3.8 Expiry is not revocation 18
3.9 Reuse without authority inflation 18
4 Package integrity 20
4.1 Signed representations and key roles 20
4.2 Inventory boundary 20
4.3 Existing Merkle rules 21
4.4 What a seal does not establish 21
4.5 Why inventory equality accompanies the root 22
4.6 Self exclusion and independent file identities 22
4.7 What the tamper cases show 22
4.8 Trust boundaries outside the directory 23
5 Migration and historical preservation 24
5.1 Why a new copy is necessary 24
5.2 Inputs and destination 24
5.3 Staging and failure 25
5.4 Relation to execution time 25
5.5 Historical metadata and fresh decisions 25
6 Evidence-bound transition protocol 27
6.1 A transition record has a bounded subject 27
6.2 Evidence has a type and a date 28
6.3 Technical findings do not supply the human disposition 28
6.4 The July HOLD and remediation example 28
6.5 Preservation and canonical comparison 29
6.6 Editorial revision is a different transition 29
6.7 Applying the protocol elsewhere 30
7 Verification and comparison 31
7.1 Current and historical evidence are not one experiment 31
7.2 Reproduction means matching the defined outcome 32
7.3 Functional comparison with v1.0 32
7.4 Source availability as a result condition 33
7.5 July 21 documentary context 34

8 Governance limits and future work 35
8.1 Human review in the proposed project profile 35
8.2 The fictional review example 36
8.3 Historical rights and publication statements 36
8.4 Limits of the demonstrated technical boundary 37
8.5 Planned v3 enterprise reproduction 37
8.6 Separate CSC research 38
Conclusion 39
References 40
Published background 40
Current v2 standards paper pair 41
V2.0 Standards Edition release 41
Planned v3 enterprise destination not published 41
Appendix A Implementation and evidence locators 42
A.1 Current implementation sources 42
A.2 Historical and documentary sources 43

1 Problem and foundations

The central problem is authority inflation: a finding made for one purpose is reused as permission for a different action. A package-selection result identifies an input. A lifecycle review may establish technical eligibility. A freeze records a bounded dataset state. None of these facts alone permits parameter updates, provider transfer, model publication or deployment. The reference controls separate these meanings so that missing or mismatched evidence does not silently become permission to continue.

The report builds on, rather than repudiates, v1.0. The earlier three-path implementation supplied reviewable corpus transformation, Access-Mode routing and contained external onboarding. TR007 provides lifecycle background; TR008 concerns interaction-role signal exchange; TR009 concerns session execution and historical CSC observations. TR010 concentrates on the implementation boundaries and reusable transition account. This progression is background, not a claim that every prior paper is executable in the compact standards selection.

Dataset lifecycle states, Access-Mode or reasoning postures, and model architecture describe different things. The reference control implementation is not itself a new trained-model architecture. Its successful synthetic verification does not demonstrate Q-Pathformer model quality, benchmark performance or CSC effectiveness.

Where this report uses **shall** in its reusable profile, the term applies to an implementation or process explicitly adopting that stated profile and scope. It does not make the profile an adopted external standard, impose a universal legal duty, or prove that every proposed requirement is implemented. The paper distinguishes implemented checks, retained executed observations, proposed project requirements and external responsibilities.

1.1 Readiness is not permission

Consider a dataset assembled from records with traceable origins. Its files are readable, the rows satisfy a schema, and a technical review finds no missing content. An operator selects it for a parameter-update experiment. These facts answer useful questions about preparation, but they do not answer whether that experiment is permitted. The organization may have approved inspection only. A prior decision may have expired, may identify a different dataset, or may concern a different operation. A successful preparation check cannot resolve these differences by itself.

This example is a statement of the problem, not an additional executed test. It explains why the consuming endpoint needs an explicit contract. A caller should not obtain a different capability merely by changing a command-line label. Equally, a package should not acquire authority from being placed in a directory whose name contains `approved`, `frozen` or `release`. Labels help locate records; a check must identify which record is consumed and what its decision actually means.

The relevant separation is not between automation and all human involvement. Automation can consistently reject a mismatched digest, an invalid signature or a scope that its endpoint

does not implement. Human review can determine intended purpose, rights, responsibility and conditions that the current runtime does not interpret. The method is useful when the boundary between these functions is explicit. It becomes misleading if a technical PASS is allowed to stand in for all of them.

1.2 The two contributions

As an implementation report, TR010 identifies concrete files and the behavior they implement. Its assertions can be compared with source code, schema constraints and named tests. This role is descriptive: a feature is not implemented because a paragraph says that a future system should have it. The report therefore records the difference between current checks and limits such as the absence of replay prevention or practitioner-credential verification.

As a reference protocol, the paper describes how a transition can be made inspectable. An adopting process identifies the input object, evidence, decision, permitted operation and resulting artifact. The transition description is reusable even when a different system has a different dataset representation or workflow. Reuse does not mean that this implementation has already verified that system, or that the July milestone sequence is mandatory for every organization.

The companion paper has a deliberately different task. TR010-R reports counts, observations, historical comparisons and source availability. It explains what an executed check establishes and which stronger conclusions are unsupported. Keeping that material in the results paper allows this method account to give enough implementation detail without repeatedly reproducing each historical report. The two papers should be read together when a claim depends on both mechanism and observation.

1.3 Relationship to the earlier papers

Table 1 identifies the conceptual inheritance. It is not a dependency installation list or a claim that every earlier source package is present in the compact selection. Bibliographic entries and publication navigation appear at the end of the paper.

Table 1 Prior work and the boundary retained in this report

Prior work	Role in the progression	Boundary retained
APOD-TR-002	Governance framework background.	Governance descriptions do not authenticate a particular decision or confer legal authority.
APOD-TR-006 Version E	Access-Mode reasoning orchestration.	Reasoning posture is not a dataset lifecycle state.
APOD-TR-006-E.2	Working-PoD artifact-state interpretation.	An artifact's completeness and inspection posture do not authorize training.

Prior work	Role in the progression	Boundary retained
APOD-TR-007	Dataset onboarding, passports and lifecycle routing.	Traceable origin and controlled movement remain distinct from permission for a named use.
APOD-TR-008	Interaction-role signal exchange.	Upstream signal artifacts and downstream processing do not create authority by convergence.
APOD-TR-009 and TR009-R	Session execution and historical CSC evidence.	Session-bound observations retain their dates, coverage and qualifications.
QPATH-ARCH-001	Multi-state model-architecture background.	The control-layer verification is not a trained-architecture benchmark.

The Access-Mode release description identifies Version E as the method paper and E.2 as an informative companion that does not extend that method. This report follows that distinction. Its new technical contract does not revise the earlier papers' meaning or replace their notices. It identifies a current implementation boundary within Q-Pathformer Enterprise. [B2, B3]

1.4 Four classes of statement

To avoid treating every sentence as a runtime guarantee, this report uses four evidence classes. An implementation statement identifies a source-consumed field or check. An executed observation refers to a retained log or report. A profile requirement expresses what an adopting process would need to do. A future proposal names work not established by the current evidence.

Table 2 Reading implementation and governance claims

Class	Example	Evidence needed
Implemented check	The verifier rejects an ALLOW with more than one scope.	Current source and applicable schema.
Executed observation	The named consolidation test passed.	Retained test entry and source identity.
Proposed profile requirement	An organization records the decision-maker's responsibility.	Adoption and actual decision records, not a schema label alone.
Future capability	A managed revocation or credential service.	Separate implementation and verification before any current capability claim.

A source inspection and a passing test are complementary, not interchangeable. Inspection can reveal an implemented duration check for which the selected test module contains no dedicated boundary assertion. A passing test can show a particular rejection without proving

that every parser input, path spelling or concurrent filesystem behavior is covered. The paper keeps both observations rather than replacing them with a single unqualified claim of compliance.

1.5 Evidence and release identities

This edition centers on the September synthetic control exercise. The earlier v1.0 execution and July v2 candidate records supply comparison and a worked release-engineering example. They are not parts of one continuous new experiment. A manuscript revision changes editorial bytes, not the date, input object or permission of a retained run.

The July source commit and candidate digest identify different objects; the September parent commit and uncommitted source capture identify another selection. A final publication PDF would have another file identity. Keeping these distinctions matters because a reader may otherwise assume that a familiar tag, folder or paper title denotes unchanged content. Section 6 explains the transition consequences, while TR010-R supplies the detailed chronology and evidence catalogue.

2 Lifecycle control model

The synthetic example can be read as one short chain:

Synthetic dataset -> one-operation authorization -> sealed package -> valid dry run and rejection tests -> verification report.

Here, one-operation authorization means one named authorized scope, not a token that can be used only once. The fixture begins after lifecycle preparation: its four records are already represented as STATE_1 with a freeze and local registry entry. The rehearsal therefore tests the selected authorization, packaging and runner boundaries, not a new traversal through every review, promotion, handoff and freeze operation.

The wider control method separates evidence, technical eligibility, human decision, signed authorization, package integrity and execution. A successful check permits only the transition that actually consumes it. It cannot supply a missing decision or retroactively approve an earlier output. A negative result may be the intended outcome of a control test rather than a failed rehearsal.

For the tested rejection paths, the runner does not enter the backend or leave a final success directory. This is narrower than saying that no filesystem activity is possible: temporary staging and bounded failure evidence may exist. It also does not prove cleanup after every possible operating-system, hardware or backend failure.

The synthetic harness blocks selected ML imports and socket connection entry points and requires the trainer's dry-run mode. These are bounded test-harness controls. They are not a general operating-system sandbox, a proof of network isolation against arbitrary code, or evidence that all production dependencies were executed.

Implementation basis: `tools/rehearse_standards_release.py`, `tools/standards_rehearsal_child.py`, `scripts/run_training_state1.py`, `scripts/run_state1_finetune_local_lora.py` and their named unit tests. Executed-result basis: the retained consolidation unit log and rehearsal JSON, not this prose or a new execution.

2.1 Selection is an input decision

The lifecycle scripts support explicit package selection and bounded latest-root searches. An explicit package path identifies the object the caller wishes to examine. A latest-root search considers packages within a particular root and applies the stage's eligibility conditions. Neither mechanism is a grant of authority. The output must record the selected package and the reasons supporting its stage-specific result.

The existing lifecycle tests exercise a newer held package alongside an older eligible one. For review, promotion and handoff, they check that the search selects the eligible package rather than treating modification order as approval. They also check that the unselected newer package does not receive that stage's success-oriented artifacts. These fixture observations

establish the tested selection behavior; they do not establish a universal global ordering or authorize a search across unrelated roots. [S4; TR010-R section 2]

The distinction is important for reproducibility. A command using `--latest` without a sufficiently specific source context can select a different object on a later day. A recorded package path, dataset version and source identity let a reviewer determine what was actually processed. Repeating the same textual command is not necessarily replaying the same evidence object.

2.2 Missing evidence remains missing

The reviewer distinguishes an absent maturity field from a mature record. The missing-maturity test supplies a candidate whose routing and content are otherwise populated, omits maturity, and expects `INSUFFICIENT_EVIDENCE` with a zero level-four-or-higher ratio. The output records the missing candidate identity. This prevents an omitted field from becoming a favorable default in that tested case. [S4]

The conclusion is not that every sparse package is unsafe or that a maturity label establishes real-world quality. It is that the technical rule consumes evidence explicitly rather than guessing it. A future reviewer may provide additional evidence or make a different authorized decision, but the old missing-field record remains a record of what was available at the time.

The same reasoning applies to other required inputs. Absence of an authorization is not equivalent to a signed DENY, but neither supplies permission to proceed through an endpoint that requires an ALLOW. An unsupported version is not implicitly translated into the current contract. A missing source package is not recreated merely because a historical report describes it. Each absence has its own diagnosable meaning and must not be hidden by a generic success status.

2.3 Technical review and human decision

The current review terminology distinguishes technical eligibility from a human disposition. `ELIGIBLE_FOR_HUMAN_REVIEW` describes passage through the applicable technical conditions. It does not assert that a person has evaluated rights, organizational purpose or downstream use. Likewise, a boolean field historically named `approved` must be interpreted in the scope of its containing stage and decision record, not as general approval of all later activity.

This discipline is not a demand to remove all automated decisions. The code can decide whether its technical predicates hold. The organization can decide whether the resulting evidence is sufficient for a named action. A clear record states which decision occurred. The paper does not transform an automated review file into a signed professional opinion, nor require an operator to infer the scope of a decision from an ambiguous filename.

2.4 Transition invariants

Table 3 consolidates the principal invariant types used in the method. Some are observed in the current fixtures; others state the interpretive rule needed when reusing the protocol. The table does not imply that one September rehearsal exercised the entire lifecycle.

Table 3 Lifecycle and execution invariants

Boundary	Required interpretation	Current evidence context
Selection to review	A selected path identifies an object, not its approval.	Latest-root selection fixtures.
Missing maturity to eligibility	Absence is not silently upgraded to maturity.	Missing-maturity rejection fixture.
Review to promotion	Consume the applicable review result and other stage conditions.	Held versus eligible package fixtures.
Handoff to freeze	A held handoff must not appear freeze-ready.	Freeze-ready output suppression test.
Freeze to registry	A held freeze does not justify registry mutation.	Unchanged-registry byte assertion.
Registry to preparation	A registry entry is a local input, not explicit training permission.	Separate v2.2 authorization checks.
Package to runner	Recheck package integrity and operation-specific authorization at use.	Valid dry run and rejection cases.
Historical object to migrated copy	Preserve the source and identify the new object separately.	Copy-migration tests and source-binding checks.

The table should not be read as a series of interchangeable PASS tokens. A freeze checks a different object and proposition from an execution-time authorization. A stage may need several inputs, and a result may be technically favorable while the requested operation remains unauthorized. This is why the artifact's scope, identity and consuming endpoint are essential parts of the record.

2.5 Positive outputs and held outputs

Output discipline is part of the control boundary. If a held path writes a file that looks indistinguishable from a successful freeze or preparation, a later consumer may treat that file as evidence of completed authorization. The reference implementation tests specific absences of success-oriented artifacts while still permitting useful diagnostic records.

Table 4 Tested failure outputs and their limits

Tested condition	Record or observation retained	Success artifact not established
Missing maturity	`state2_review.json` with insufficient-evidence result.	No technical eligibility from the absent field.
Held explicit handoff	Handoff decision records the hold.	No `freeze_ready_manifest.json`.

Tested condition	Record or observation retained	Success artifact not established
Held freeze	Returned decision and metrics.	No new freeze directory or frozen dataset.
Held registry update	Returned decision; registry bytes compared unchanged.	No registry mutation or success update artifact.
Wrong runner operation	Expected rejection evidence.	No backend entry or final success run directory in the tested case.
Unsupported run-log route	Expected route rejection.	No final run directory in the tested case.
Tampered sealed package	Inventory mismatch rejection.	No successful dry-run artifact.

A diagnostic file is not prohibited simply because it is an output. Its naming and contents need to make the hold clear. Conversely, a correct error message alone is insufficient if a success artifact has already been left behind. The tests therefore examine both the reported result and selected filesystem consequences. They do not claim that every temporary file or every possible interrupted operation is covered.

2.6 Freeze and registry are not operation grants

A freeze records a particular dataset state and its supporting material. Registry handling records an identified lifecycle object. These functions are valuable for tracing a prepared dataset, but they do not answer every question about future use. For example, an inspection-oriented dataset can be faithfully frozen without being authorized for parameter updates. A registry entry can accurately describe that freeze without conferring rights to transfer it to a provider.

The preparation endpoint therefore consumes explicit authorization separately. Its local registry resolution and source digest checks establish the expected binding inputs for that decision. The runtime then checks its own package and authorization boundary. This layering does not make the supplied registry a live authority service; the limits of that design are set out in section 3.3.

2.7 What the demonstration omits

The four synthetic records let the rehearsal test a controlled route without importing historical research papers or client material. They are deliberately small and already supplied with STATE_1 and registry fixtures. Their role is to expose the endpoint boundary, not to simulate enterprise diversity or statistically validate governance performance.

Lifecycle unit tests add focused cases; they do not establish that every stage was executed consecutively on one real dataset. TR010-R reports their grouping. Full enterprise reproduction remains planned v3 work.

3 Authorization contract

3.1 Decision and scope

Under `qpath_training_authorization_v2.2`, an `ALLOW` artifact contains exactly one explicit authorized scope; a `DENY` artifact contains no authorized scopes. Scope values must be known, exactly spelled and non-duplicated. The current verifier rejects unsupported schema versions, including v2.1. It is not an automatic legacy-contract dispatcher.

Table 5 Scope vocabulary and endpoint capability

Scope vocabulary	Current endpoint capability
`validation_parameter_update`	Enforced scope for current preparation, copy-migration and local research-runner endpoints.
`production_training`	Named vocabulary; no implemented production-training endpoint is established by this standards exercise.
`external_provider_training`	Named vocabulary; no implemented external-provider endpoint is established.
`model_release`	Named vocabulary; no implemented publication-authorization endpoint is established.
`deployment`	Named vocabulary; no implemented deployment-authorization endpoint is established.

A valid signature on a DENY records a denial; it does not permit the requested action. A valid ALLOW for a different known scope also does not satisfy the current local endpoint. The contract does not implement single-use consumption, a replay counter or a live authorization-revocation service. Consequently, one scope shall not be described as one permitted execution.

The creation command requires explicit acknowledgment of a human decision for ALLOW. That acknowledgment is a supplied assertion, not independent verification of the person's identity, organizational power or professional qualifications. Required reviewer, source and rationale fields preserve decision context; their presence does not establish that the stated facts are true.

3.2 Identity and time

The verifier requires the expected structure and compares the artifact's dataset version, source-dataset SHA-256 and registry-entry identity with values supplied by the consuming endpoint. It checks signature metadata, an external public-key fingerprint, the signature and the decision/scope contract. These checks bind the authorization to specified recorded identities; the origin and current authority of the external trust inputs remain separate responsibilities.

The timestamps are parsed as timezone-aware values. The review time must equal issuance time. `not_before_utc` cannot precede `issued_at_utc`; expiry must follow the start of the valid interval. The verifier rejects `expires_at_utc - not_before_utc` greater than 31 days. The interval includes its start and excludes its expiry: `not_before <= now < expires_at`. This is not a claim that every possible future start must lie within 31 days of issuance. The creator's narrower duration option accepts 1-744 hours, with a default of 168 hours from issuance.

The existing named boundary test covers inclusive start and exclusive expiry. The 31-day maximum is implemented in the inspected verifier, but no dedicated over-limit rejection assertion was identified in the two inspected authorization test modules. Source inspection establishes that the check exists; it does not manufacture an additional executed test result.

3.3 What each endpoint actually consumes

Table 6 Endpoint bindings and limits

Endpoint	Binding behavior	Important limit
Preparation: `run_training_state1.py`	Reads a supplied local registry, resolves the selected STATE_1 entry and frozen inputs, hashes the frozen source dataset and checks authorization against those identities before final package construction.	A supplied local registry is not a live external authority/revocation service.
Copy migration: `migrate_training_package_authorization_v2.py`	Reads the old package manifest and explicit source dataset, hashes the source file, validates a fresh authorization and creates a separately sealed copy.	This does not establish fresh registry service consultation or grant rights retroactively.
Local research runner: `run_state1_finetune_local_lora.py`	Verifies the sealed package and authorization against recorded manifest identities using external public keys before the selected run.	It compares the authorization's source digest to the sealed manifest's recorded source digest; it does not reopen the original frozen source dataset or consult a live registry at execution time.

The training-file bytes are protected as package members. That integrity protection does not independently prove the semantic derivation of every training row from the source dataset.

Historical `fine_tuning_allowed` flags, whether true, false or absent, are classified as legacy context rather than promoted into the current explicit authorization. A separate new decision is required. The reference implementation does not infer permission from a prior run, model label, lifecycle status or a successful migration.

3.4 Recorded policy versus enforced policy

The current authorization contract does not contain structured checks for a permitted model identifier, execution environment, maximum training steps, practitioner credentials or arbitrary free-text conditions. A backend profile may be sealed, and the runner validates configuration and positive numeric overrides, but those facts are not a legal-policy interpreter or proof of organization-approved hyperparameter bounds.

Such conditions may remain part of the responsible organization's documented decision and procedures. They must be described as external policy unless a matching implemented field, consumption point and evidence record is identified. A broader future service requires its own defined scope and verification; it is not delivered merely by retaining all five vocabulary values.

Implementation basis: `scripts/training_authorization.py` (`validate_training_authorization`, `_validate_scope_contract`, `require_local_lora_scope`); `scripts/create_training_authorization_v2.py`; the three endpoint scripts; and the existing authorization schema/tests.

3.5 Structure before interpretation

The schema and verifier impose different but complementary constraints. A JSON Schema can reject a missing member, an unexpected shape or an invalid enumerated value. The runtime verifier also checks cross-field relationships and values expected by the consuming endpoint. A schema-valid document is therefore not automatically executable. In particular, the endpoint's required scope and expected dataset identities must still agree with the signed record.

The verifier uses exact-key checks for accepted objects. An extra field describing a proposed policy is not silently promoted into a machine-enforced rule. Adding such a field requires an intentionally versioned contract and consuming implementation. This reduces the ambiguity in which a human sees a condition in JSON and assumes that every runner must be enforcing it.

Table 7 Authorization fields and the claims they support

Element	Implemented interpretation	Not established by that element
Schema version	Selects the accepted v2.2 structure.	Automatic conversion of older contracts.
Decision and scopes	ALLOW has one scope; DENY has none.	Single-use consumption or support for all named endpoints.
Dataset binding	Compared with expected version, digest and registry identity.	Truth of all source provenance or current external registry status.
Human-review context	Required recorded reviewer, source, rationale and timing.	Credential verification or legal competence.

Element	Implemented interpretation	Not established by that element
Validity	Parsed interval, relationship checks and current-time boundary.	Trusted timestamping or revocation after issuance.
Authentication	Signature representation and external-key verification.	Organizational authority derived from key possession.

The field list explains why a record may be authentic and still fail. It may contain a valid denial, an expired permission or a permission for a different dataset. In each case, signature success is a step in the decision, not its conclusion. The verifier is intended to preserve these differences rather than classify every authentic document as allowed.

3.6 Decision creation and acknowledgment

The creation command takes the identified source dataset and decision context, computes the source binding, constructs the accepted contract and signs it. For ALLOW it requires the explicit human-decision acknowledgment. It refuses to replace an existing output file. These are useful protections against accidental implicit permission or overwriting a previous decision artifact.

The acknowledgment does not create an independent identity service. A person or process supplying the command can assert reviewer information; the current tool does not consult a practising-certificate registry, verify an employment role or establish that the stated organization delegated the decision. Those are external responsibilities of a process that uses the tool. The paper does not claim otherwise simply because a reviewer name appears inside signed data.

The distinction also affects demonstrations. Disposable fixture keys and a synthetic authorization test the technical format and control flow. They are not evidence of a real organizational decision to use historical or client data. The resulting dry-run report is valuable precisely when it remains labelled as evidence of those bounded checks.

3.7 Authentic but unacceptable cases

Negative cases should not be limited to malformed JSON. A well-formed and correctly signed object can be the wrong input for an operation. The current tests include signed DENY, time boundaries, binding mismatches, endpoint-scope mismatch and reused trust-role keys. This makes it possible to distinguish authentication from acceptance in observable behavior.

Table 8 Interpreting selected rejection classes

Case	Why authenticity is insufficient	Consequence in the tested contract
Signed DENY	The decision records non-permission.	Rejected for an operation requiring ALLOW.

Case	Why authenticity is insufficient	Consequence in the tested contract
Future start	The valid interval has not begun.	Rejected before `not_before_utc`.
Reached expiry	Prior permission does not renew itself.	Rejected at and after expiry.
Different dataset identity	The permission concerns another bound object.	Expected binding mismatch is rejected.
Different known scope	Vocabulary membership does not change the endpoint.	Local endpoint mismatch is rejected.
Older schema identifier	Similar-looking fields are not the supported contract.	No implicit v2.1 dispatch.
Multiple ALLOW scopes	A broader grant is outside current cardinality.	Rejected rather than choosing one silently.

These cases do not enumerate every possible rejection or prove exhaustive parser security. They show why an implementation account needs the relationship between fields and consumption points, not just a sample signature envelope. The results paper preserves the number of named executed tests rather than counting each row in this explanatory table as another test.

3.8 Expiry is not revocation

An expiry bound provides a scheduled end to validity. Revocation would address a permission that should cease before that end, for example because an organizational decision changed. The current verifier does not consult a live revocation infrastructure. It checks the supplied record and trusted-key input under its current-time rule.

This difference limits how the implementation can be reused. An organization may have external procedures to replace trust inputs or stop a run, but those procedures are not established by the current JSON contract. The report therefore treats managed revocation, key rotation and distribution as future or external controls. It does not describe a 31-day ceiling as complete permission-lifecycle management.

The same distinction applies to repeat execution. A still-valid, matching ALLOW may be presented again because the contract has no consumption counter. A record of one authorized scope does not establish that the permission is used once and then exhausted. Any single-use design would require additional state and its own failure and recovery semantics; it is not implied by the present terminology.

3.9 Reuse without authority inflation

The authorization artifact can be an inspectable technical carrier for a recorded decision. Its reuse is defensible only when the adopting process preserves the decision's purpose and the

endpoint's limits. A consumer cannot infer that `production_training` is supported because the vocabulary contains that string, or that a legal condition is enforced because related prose accompanies the package.

The proposed reference profile calls for a clear mapping from each required condition to an actual mechanism or responsible human process. Conditions without an implemented consumer remain external. This is a documentation and accountability discipline, not a claim that the current source already supplies every production feature. It permits the reference method to be useful while making the gaps visible to reviewers.

4 Package integrity

4.1 Signed representations and key roles

Authorization and package sealing use Ed25519 with distinct key roles. The preparation, migration and local runner reject use of the same public-key fingerprint for both roles. Trust inputs are supplied outside the package. Distinct keys separate technical roles; they do not establish organizational independence, qualified legal review or secure key custody by themselves.

For authorization, the signed representation is a deep copy of the JSON object with only `authentication.value` removed. For a package seal, only `signature.value` is removed from the copied seal object. All other accepted fields, including the relevant signature metadata and public-key fingerprint, remain in the signed representation.

The implementation serializes that object with Python JSON settings `sort_keys=True`, `separators=(",", ":")` and `ensure_ascii=False`, then encodes the result as UTF-8. The method identifier is `qpath-json-sort-keys-v1`. It is a project-specific representation, not a claim of RFC 8785 canonicalization or additional Unicode normalization. A conforming independent implementation would need to reproduce these exact accepted-value and serialization semantics.

The public-key fingerprint is the lowercase hexadecimal SHA-256 of the raw Ed25519 public-key bytes, not of PEM text. The raw signature is Base64 encoded for the artifact. Verification performs strict Base64 decoding, checks the signature length and verifies the bytes against the supplied public key. A matching fingerprint and signature establish a cryptographic relationship, not the legal identity or authority of a named signer.

4.2 Inventory boundary

The package inventory describes regular-file content beneath the resolved package root. Each entry has exactly `path`, `size_bytes` and `sha256`. Paths are relative and expressed with `/`; entries are sorted by the UTF-8 bytes of the path. Each file digest is SHA-256 over the actual file bytes, and size is recorded in bytes.

The canonical self-exclusion is exactly the root-relative member `package_seal.json`. It is not a wildcard for every file with that basename. A nested `package_seal.json` is therefore not excluded by that rule. Empty directories do not appear in the inventory, and a package with no sealable files is rejected.

The inventory walk rejects entries it encounters as symbolic links. This finding shall not be broadened into verified rejection of every Windows reparse-point type or every possible alias used to name the root: the source resolves the root before traversing it. The independently documented exporter link-capability test has its own observation history. Neither a clean scan nor source inspection substitutes for executing that assertion on a capable system.

4.3 Existing Merkle rules

Let `H` be SHA-256, let `p` be the relative path string, let `n` be its recorded non-negative byte length, and let `d` be the 32 raw bytes obtained by decoding the recorded file SHA-256 hexadecimal value. The implementation's leaf is:

```
leaf = H(b"qpath-file-v1\x00"
         + UTF8(p)
         + b"\x00"
         + ASCII(decimal(n))
         + b"\x00"
         + d)
```

The `\x00` notation denotes a zero byte, not four printable characters. Inventory leaves enter the tree in the path order described above. For each pair of 32-byte child hashes, the internal-node rule is:

```
node = H(b"qpath-node-v1\x00" + left + right)
```

At each level with an odd number of entries, the last entry is duplicated before pairing. A one-leaf inventory has that leaf as its root. An empty inventory is rejected rather than assigned a special root. The final root is represented as lowercase hexadecimal and identified by `qpath-merkle-sha256-v1`.

The seal schema is `qpath_package_seal_v2.0`. Verification checks the expected manifest/dataset-version/registry binding, signature and supplied key, inventory structure and ordering, exact equality with the rebuilt regular-file inventory, and the recomputed Merkle root. Changed, added or removed inventory members therefore alter the compared package representation. The canonical seal file is excluded from the member inventory to avoid circularity; its accepted metadata is covered by the separate signature representation.

The existing sealing tests cover a deterministic three-file inventory/root, valid versus wrong-key signatures, manifest-binding mismatch, changed/added/removed members and seal-metadata tampering. They do not supply an independently calculated literal known-answer vector. This paper reports their scope without inventing one or claiming a new cryptographic assurance engagement.

4.4 What a seal does not establish

A valid seal does not establish permission to train, content safety, correct labels, provenance truth, confidentiality, availability or a model-quality result. External configuration, key custody and selection of trusted public keys lie outside the regular-file inventory. Local creation-time metadata is not an independent trusted timestamp. Verification at one point does not eliminate time-of-check/time-of-use risk under concurrent changes or a compromised execution environment.

Implementation basis: `scripts/package_sealing.py` (`build_package_inventory`, `_leaf_hash`, `calculate_merkle_root`, `canonical_package_seal_bytes`, `validate_package_seal`); `scripts/crypto_keys.py`; `scripts/training_authorization.py`; and `tests/unit/test_package_sealing_v2.py`.

4.5 Why inventory equality accompanies the root

A Merkle root gives a compact commitment to the ordered leaf representation, but a verifier still needs to know what was committed. The inventory supplies paths, sizes and content digests. The current verifier checks the inventory's structure, order and equality with a rebuilt view of the package, as well as the root and signature. These checks connect the compact value to the files the runner is about to consume.

Checking only a file's content digest would not establish that all required or unexpected members were accounted for. Checking only a manifest file would leave other package content outside that comparison. The regular-file inventory makes additions and removals part of the represented object. It does not inventory empty directories or external dependencies, and it does not turn a package directory into a complete system-state snapshot.

The path encoding is part of the commitment. A differently named member with the same bytes produces a different leaf because the relative path is included. Size is included as a decimal ASCII representation, and the content digest enters as decoded bytes rather than its printable hexadecimal string. These details matter for an independent implementation; changing them creates a different root rather than another spelling of the same contract.

4.6 Self exclusion and independent file identities

A file cannot straightforwardly contain a digest of its own final bytes without a separately defined construction. The package design avoids that circularity by excluding the root seal file from the member inventory while authenticating its accepted metadata through the seal signature. The exclusion is narrow and explicit, not a general exception for convenient files.

The same principle applies to manuscript publication. A paper's final DOCX or PDF hash belongs in an external revision or publication record, not in a paragraph that changes each time the hash is inserted. A runtime package seal, an editorial identity record and an archive checksum serve different purposes and should not be called the same evidence object.

When a historical source ledger identifies an earlier paper, a later editor preserves that record and adds a new mapping. Updating the old hash to match a new file would erase the distinction between original and revised objects. In the present source reconciliation, the E01 ledger path contains a later manuscript revision; the results paper documents that difference instead of treating it as either an exact match or a runtime failure.

4.7 What the tamper cases show

The sealing tests examine changed, added and removed package members, wrong verification keys, manifest-binding mismatch and altered seal metadata. The three-file inventory test exercises deterministic ordering and odd-leaf handling. These are concrete checks against the implementation's declared representation. They are not a cryptographic proof of every code path or an independently certified implementation of the algorithms.

The absence of an independently calculated literal known-answer vector is especially relevant for interoperability. Repeatedly computing a root with the same implementation checks

consistency but may not reveal a shared mistake in both calls. The compact paper therefore states the actual byte rules and the present test scope. It does not add a new vector and misattribute it to September, or modify source code to make a documentation claim easier to defend.

A failed integrity check identifies a mismatch, not necessarily malicious intent. A text editor, packaging tool or accidental file addition can also change bytes. The appropriate conclusion is that the current package no longer matches the verified representation. Determining cause and authorizing a new package are separate actions. An operator should not normalize a mismatch away unless a separately defined and authorized contract specifically permits that transformation for that object.

4.8 Trust boundaries outside the directory

The verifier receives public keys from outside the package so that an untrusted package cannot define its own entire trust basis merely by including a key file. The caller still needs to select the correct key and establish why it is trusted. Distinct authorization and sealing keys reduce technical role conflation, but one person can control both; the code does not prove separation of organizations or duties.

Private-key protection, rotation, revocation, recovery and distribution are not solved by saving a public-key fingerprint. The local key-generation utility and disposable rehearsal keys demonstrate the reference mechanism. They do not establish hardware-backed custody, enterprise key policy or a production signing service. The seal also supplies integrity rather than encryption: sensitive content requires separate confidentiality controls.

There is a time boundary as well as a directory boundary. Files can change between a check and a later use unless the operating context prevents or detects that change. The current tests establish the selected verification order and rejection behavior; they are not proof against arbitrary concurrent filesystem mutation or a compromised host. The reference profile must state these assumptions when it is adapted to another execution environment.

5 Migration and historical preservation

Migration creates a new artifact without rewriting the historical source package. The current migration command requires the local endpoint scope, an explicit source dataset, a fresh authorization and distinct signing roles. It copies the old package into a separate staging location, removes an inherited seal only from that copy, writes the updated manifest and migration record, adds the new authorization and creates a new seal before moving the copy to its final destination. An existing destination is not silently overwritten.

This process preserves the old object as historical evidence. It does not convert old metadata into new permission, change what an earlier run was authorized to do, or resolve underlying legal rights. Rehashing the explicit source file also does not independently prove every semantic transformation used to produce the old training records. Those claims require their own evidence and remain distinct from copy integrity.

5.1 Why a new copy is necessary

Historical artifacts may contain lifecycle metadata or signatures whose meaning differs from the current contract. Editing those files in place would change the object that an earlier report identified. Even if the edited copy passed current checks, the history would no longer support the claim that the earlier bytes remained intact. Copy migration preserves that distinction by making a new output object explicit.

The new copy is not merely a relabelled old permission. It requires a fresh decision represented under the supported contract, an identified source dataset and a new seal. The migration record links the copies without claiming that the current authorization existed at the time of the historical run. This is the non-retroactivity rule: a new decision may support a new operation but cannot rewrite the authority of a past operation.

5.2 Inputs and destination

The migration command consumes an existing package and an explicitly identified source dataset. It requires the files needed to interpret that package and binds the new authorization to the source file's actual digest and the recorded dataset and registry identities. It enforces the local research scope before proceeding. The destination is a separate versioned package location; an existing destination is rejected rather than silently reused.

Table 9 Migration objects and their treatment

Object	Treatment	Reason
Original package	Read and preserve.	Retains the object identified by earlier evidence.
Explicit source dataset	Read and hash.	Supplies the source binding for the new decision.
New authorization	Validate under current v2.2 rules.	Legacy metadata is not promoted to permission.

Object	Treatment	Reason
Inherited seal in the copy	Remove from the staged copy before resealing.	An old signature does not authenticate modified output.
New manifest and migration record	Write in the new copy.	Identifies the current binding and historical relationship.
New package seal	Create with the separate sealing role.	Authenticates the resulting member inventory.
Final output directory	Publish locally only after the staged construction completes.	Avoids presenting an incomplete staged copy as a successful package.

The word publish in this table means exposing the final local directory within the migration operation. It does not mean a GitHub publication, release asset upload or permission to distribute the data. Those actions are outside the demonstrated migration endpoint.

5.3 Staging and failure

The command builds in a temporary sibling directory before moving the completed copy into its final location. This permits failure handling to distinguish an incomplete build from a finished package. It also means that the phrase no outputs must be used carefully: staging is an expected part of construction, even when a later check prevents the final object from being created.

The implementation and tests support source preservation and selected failure paths. They do not prove universal rollback under a power failure, storage corruption or every operating-system exception. A reusable protocol identifies the intended final artifact and the evidence of a completed transition rather than assuming that any directory bearing a familiar name represents success.

5.4 Relation to execution time

After migration, the local runner still verifies the resulting package. Migration success is not a token that bypasses the runner's checks. The runner consumes the sealed manifest and current authorization binding using external public keys. The source file used during migration is not reopened by the runner solely because its path appeared in an earlier record.

This design gives different stages different responsibilities. Migration establishes the new copy's source-file binding at construction. Sealing protects the resulting package members. Execution-time verification checks the supplied package against that signed representation. None of these independently proves the semantic quality of all transformations or the truth of a historical rights assertion. Those claims need their own sources.

5.5 Historical metadata and fresh decisions

Legacy flags are useful for explaining what an old package recorded. Their interpretation must remain dated. A historical `fine_tuning_allowed` value does not contain all the current

decision, scope, time and binding semantics. Treating true as automatic ALLOW would bypass those conditions; treating a missing value as an implicit default would do the same in a different way.

The current contract instead classifies the old flags and requires explicit authorization. This preserves the original record without making it govern an operation it did not describe. A reviewer can compare old and new metadata, but the comparison is explanatory evidence, not a retroactive change to the old package's authority.

6 Evidence-bound transition protocol

The reusable contribution is a way to specify a transition without merging technical findings and human authority. For a defined transition, record the exact input object, the evidence consumed, the implemented acceptance conditions, the responsible decision, the scope of any required permission, the allowed output and the failure disposition. A new transition or changed object needs its own applicable evidence; a prior successful check is not transferable merely because its label looks similar.

Table 10 Elements of the reference transition protocol

Transition element	Question answered
Object identity	Which exact dataset, package or candidate is under consideration?
Evidence	Which retained observations or records support the technical finding?
Conditions	Which checks are implemented, and which obligations remain external?
Decision	What was decided, by whom or by which explicitly identified technical process?
Permission	Which named action is authorized, if any, and under which recorded limits?
Output	Which artifact may be created after these conditions are satisfied?
Failure	What is held or rejected, and what bounded diagnostic evidence is retained?

These elements describe a reference protocol, not a new gate schema or automatic hierarchy. The July milestone sequence remains a dated worked example. The retained M12 review raised three findings; a bounded human authorization permitted the specified M09 corrections, followed by M10 checks and an M11 rebuild. The corrected report records readiness for M12 reconsideration, not a new M12 approval or public release. Repeating that history in a paper cannot provide the missing disposition.

A separately recorded technical check or repeated read-only export may strengthen an internal account. It is not, without further evidence, an organizationally independent assurance engagement. Historical report titles are preserved, while the surrounding interpretation identifies the actual strength of the evidence.

6.1 A transition record has a bounded subject

The protocol starts with the subject of a decision. A source tree, exported candidate, frozen dataset, training package and final PDF are different objects even when they share a project

name. An adequate transition account identifies the relevant object using the representation applicable to it: a commit and captured file set, a file digest, an inventory or another explicitly defined binding.

The binding must be strong enough for the claim. A Git tag is a useful locator, but its name alone does not show hosting-enforced immutability or the identity of an attached ZIP. A folder name is useful navigation, but it is not a content digest. A source manifest describes its listed files, but it does not establish that absent files were reviewed. The protocol preserves these distinctions rather than choosing one convenient identifier for all purposes.

6.2 Evidence has a type and a date

Evidence records need their own identity, date and scope. A retained unit log supports what its named test entries report. A source file explains a mechanism. An operator-supplied console transcript has a different capture basis from an automatically retained subprocess report. A historical narrative can report a useful finding while the full underlying dataset remains outside the current selection.

The protocol does not discard weaker evidence merely because it is not an independent replay. It labels the strength of the claim. For example, the retained July source reports the historical Path 3 discrepancy. The compact edition preserves that negative finding and identifies its source, while making clear that the complete bundle was not newly reopened. This is more informative than either erasing the finding or claiming an unperformed independent verification.

6.3 Technical findings do not supply the human disposition

A technical report can identify conditions that passed, findings that remain open and the object that was checked. The responsible human process decides whether that evidence supports the requested release or operation. The current artifact formats do not make every such decision cryptographically or legally equivalent. A recorded human release authorization must be interpreted in its own scope, not inferred from a technical report's favorable status.

This is particularly important for a remediation loop. Permission to correct three specified files and rerun checks is not permission to commit, upload, issue a licence or publish a paper. Completing the authorized technical work may make the candidate ready for another decision, but it does not expand the original permission. The protocol records the endpoint of that authorization so that the next action remains explicit.

6.4 The July HOLD and remediation example

The July M12 technical review identified three findings: custom SPDX metadata, remote pipe-to-shell installer guidance and unpinned GitHub Actions references. The retained authorization permitted a bounded return to M09 for those findings, followed by M10 checks and an M11 rebuild. This example demonstrates the separation between a finding, permission to remediate and a later release decision. [H3-H5]

The corrected M10 report records 24 tests, 23 passes and one environment-guarded skip, alongside its conformance and finding-specific checks. The corrected M11 report records the rebuilt candidate and two separately recorded read-only verifications. Its status is `READY_FOR_M12_RECONSIDERATION`. The next human review is not authorized in that report. The compact paper preserves that exact boundary rather than converting readiness into PASS.

The candidate's inventory, build report and checksum file were written in a defined order. The inventory listed the pre-record file set; the checksum file excluded itself; the final candidate included the checksum file. The resulting counts differ for an explained reason. TR010-R retains the 348, 350 and 351 counts and the final historical digest. The present manuscript's file identity is not substituted into any of those records.

6.5 Preservation and canonical comparison

The historical export process distinguished byte-preserved evidence from active text subject to a specified normalization. That is an object-specific rule, not a general licence to rewrite inputs. The M09 equivalence resolution likewise retained physical source bytes while recording the sole CRLF-to-LF comparison used for six entries. Canonical equivalence under that rule does not make physically different files byte-identical. [H2, H6]

This distinction prevents two opposite errors. One is declaring an explained line-ending comparison to be a byte-integrity failure when the recorded rule intentionally compares a canonical representation. The other is applying that rule to unrelated material whose contract requires exact original bytes. The historical freeze-bundle discrepancy cannot be resolved by borrowing a normalization rule from a different standards package.

When an adapting process permits a transformation, the transformation and its scope need to be recorded before the resulting identity is interpreted. If the source contract is unknown, the protocol retains the uncertainty instead of inventing a rule that produces a desired match. A reproducible export is valuable only when the treatment of each class of material remains inspectable.

6.6 Editorial revision is a different transition

Revising a paper is a documentary transition. Its input is the earlier manuscript; its output is a new version with a section map, changed prose and a new file identity. It does not modify the experiment described in the paper. The revision record should therefore say which statements were clarified, which sections moved and which evidence remained unchanged.

The E01 exception illustrates this distinction. An older ledger identifies a main-paper revision, while the file now present at that recorded path has different bytes. The new editorial crosswalk records both identities. It does not overwrite the earlier ledger, infer a runtime failure or use unchanged code hashes to claim unchanged manuscript bytes. After final rendering, the new paper pair receives its own external identity record.

Publication selection is a separate transition. It identifies the DOCX, PDF, evidence companions and source files selected for distribution. A revised manuscript is not

automatically part of a strict export allowlist. Local evidence availability does not, by itself, establish which E-group records are distributed with this edition.

6.7 Applying the protocol elsewhere

An organization can use this transition account without adopting the project's historical milestone names. It would identify its own bounded objects, evidence requirements, responsible decision process and output conditions. It would also state which checks are automated and which depend on external policy or professional review. Reuse requires this mapping rather than a simple declaration that the project is standards aligned.

The account is deliberately not a new certification scheme. It supplies an inspectable structure for claims and decisions, not accreditation, assessor competence criteria or a regulator's approval. Its value to standards exploration is that another implementation can be compared against explicit semantics and evidence limits. Its value does not depend on treating the reference implementation as production complete.

7 Verification and comparison

The current standards evidence is the retained 5 September consolidation suite and command-line rehearsal. The suite lists 48 tests: 47 passed, one was skipped and none failed. The groups contain eight exporter, seven lifecycle, five sealing, seven private-document allowlist, fifteen single-scope and six authorization/migration tests. The rehearsal matched nine expected outcomes: five exit-0 operations and four intentional exit-1 rejections. These are not four failed overall rehearsals.

The runtime source map records eleven files; a separate 6 September read-only comparison found all eleven hashes equal to the current local copies. That comparison is not a new execution or a complete attestation of interpreter, operating system, dependency supply chain or all repository files. The parent commit and the captured uncommitted bytes must remain distinguishable.

The earlier full-suite symbolic-link skip remains in its original result. A separate targeted recheck also skipped, and an operator later supplied a one-test pass in 0.010 seconds after reporting Developer Mode and an elevated PowerShell session. The supplied record does not establish an exact execution timestamp or a non-elevated pass. These observations do not retroactively create a 48-pass suite. TR010-R places this history and the capability explanation in its requirements appendix rather than expanding the abstract.

Historical v1.0 material demonstrates earlier execution continuity, not new v2 model effectiveness. The reported Path 1 data distinguish 16 source documents, 846 extracted signals and 1,241 dataset rows, comprising 1,117 training and 124 validation rows. The existence of validation rows does not establish a recorded held-out validation loss. The tiny four-record Path 2 training example does not establish generalization, and the runtime emitter's below-threshold HOLD is a separate outcome from source-first LoRA training.

Path 3 remains contained onboarding evidence. The historical ingestion report and the later source-reported freeze-bundle hash discrepancy both remain relevant; canonical text equivalence from a different standards package cannot repair that discrepancy. No historical package is silently regenerated to make the comparison cleaner.

The historical CSC figures remain profile-specific, qualified observations. Finance uses 16 of 60 metrics; the other listed profiles use 12 of 60. Those fractions are coverage measures, not calibrated confidence intervals or direct-telemetry percentages. TR010-R retains each historical composite, level qualification and source limitation. This standards exercise creates no new CSC score and does not blend old and new records into a new rating.

7.1 Current and historical evidence are not one experiment

Table 11 fixes the primary evidence periods. The current synopsis is September-first; the historical figures are retained rather than relabelled. Detailed source identities and result tables belong to TR010-R.

Table 11 Evidence periods and permitted conclusions

Period	Retained observation	Limit
April v1.0 working set	Corpus, lifecycle, local LoRA and qualified CSC observations reported by the retained July comparison.	Not a new v2 replay or model-quality comparison.
July corrected v2 candidate	M10 records 24 tests, 23 passes and one skip; M11 records 351 final regular files.	Historical candidate only; no new corrected M12 disposition.
September 5 standards controls	48 listed tests with 47 passes and one skip; nine expected CLI outcomes matched.	Four-record synthetic fixture; no model training.
Later link observations	Separate targeted skip and operator-reported single-test pass.	No retroactive all-pass suite.
September 6 manuscript reconciliation	Source identity and editorial comparisons.	Not an additional runtime experiment or publication attestation.

7.2 Reproduction means matching the defined outcome

The valid rehearsal route is a dry run, and the invalid routes are expected to reject. A reviewer reproduces the defined behavior by checking both kinds of outcome, not by trying to make every subprocess exit successfully. The retained rehearsal has five expected exit-0 steps and four expected exit-1 steps. Removing the negative cases would reduce the demonstrated boundary rather than improve its result.

Repeated executions may produce different disposable keys, signatures, identifiers and timestamps. These differences do not necessarily indicate a different control contract. A useful comparison records the exact source selection, environment and expected outcome for each step. It also retains every new failure or skip rather than copying an old success status into a new report.

The current source map contains eleven implementation, harness and backend-profile files. Matching those hashes connects a retained report to that subset; it does not attest every package in the environment or every file in the repository. The broader captures identify the uncommitted working selection beyond its parent commit. These records should be considered together without implying a trusted timestamp or independent assurance service.

7.3 Functional comparison with v1.0

The main comparison concerns control semantics. V1.0 supplied a three-path lifecycle with populated examples and recorded local model execution. The current v2 controls make explicit which authorization and integrity checks precede a named local operation. They also

demonstrate selected negative-output and copy-migration behavior. This is not a comparison of two trained models on a common prediction task.

Table 12 Functional comparison and its interpretation

Area	Historical v1.0 role	Current v2 role
Corpus path	Deterministic document and dataset preparation with a recorded local LoRA example.	No new corpus training; source and permission boundaries are illustrated synthetically.
Access-Mode path	Source-first lifecycle and separate runtime-emitter HOLD examples.	Explicit endpoint-bound authorization and failure semantics.
External adapter path	Contained onboarding with both positive and discrepant historical observations.	No automatic promotion or training authority from an ingestion result.
Permission	Lifecycle and historical metadata supplied context.	Separate signed v2.2 decision consumed at named endpoints.
Integrity	Earlier manifests and hash records.	Complete regular-file inventory and separate seal, checked before the selected run.
Migration	Earlier records remain identifiable.	New decision and sealed copy without retroactive changes.

A lower training loss in one historical loop would not demonstrate stronger governance. A larger test count in a later suite would not quantify twice the security. A smaller selected repository would not prove a smaller model. These are different measures with different denominators. The paper avoids turning them into a common optimization score without a defined comparison experiment.

7.4 Source availability as a result condition

The local E-group crosswalk contains 28 groups and 62 file references. All recorded paths were present at the 6 September comparison, with 61 exact recorded-identity matches and one separately qualified manuscript revision at E01. These counts describe the inspected local selection, not proof of the contents of a distributed archive.

The original ledgers remain dated records. The current manuscripts have their own identities, and release distribution is a separate selection. A reviewer who receives only the two papers cannot be assumed to have the full source, dependency set, original v1.0 datasets or July frozen tree. TR010-R therefore distinguishes locally retained records, external historical inputs and selection-dependent public availability.

The private workspace verifier is not a generic validator for a two-PDF public pack. It checks a particular working-draft allowlist and documentary set. The runbook describes checks in a complete prepared local workspace; those checks require the corresponding scripts and

supporting files, not only the two papers. This distribution boundary does not justify weakening a checker or claiming an unperformed run.

7.5 July 21 documentary context

The current history notice and original main manuscript describe a 21 July rights/provenance review as pre-rebuild supporting documentary evidence. It is distinct from the corrected 18 July M11 candidate. Proposed canonical-copy selection, Path 1 remapping and regeneration are not reported here as an executed rebuild or a new release disposition. The detailed original file-level review set was not established in the inspected controlled repository; its summary is therefore attributed to those documentary sources rather than independently revalidated. No practitioner legal opinion, replacement licence adoption or publication authority follows from its internal review labels.

8 Governance limits and future work

Technical service can inventory identified materials, preserve provenance, record review outcomes, enforce implemented permission fields and produce verifiable execution evidence. It cannot infer the truth of legal rights or professional competence from a valid signature alone. Qualified legal review, where required by the adopting organization's policy for identified legal questions, and organizational authorization of a named operation remain different human responsibilities.

The illustrative legal-review material remains `HOLD_PENDING_REAL_REVIEW` and `NOT_AUTHORIZED_FOR_REAL_OPERATION`. The current runtime does not consume that practitioner-review record or authenticate professional credentials. Its presence is neither an actual lawyer-reviewed rights schedule nor a universal legal mandate. The compact paper must not present automated AI legal approval as an implemented or approved feature.

Historical CSC names are retained when identifying original artifacts. The supplied reference review records the v1.0 enterprise usage as "Controlled Signal Certificate", derived from the Access-PoD "Compliance Star Certificate". These names identify historical sources; they are not renamed schemas or new certification claims. The proposed CSC Assurance Record and governance-snapshot contract are separate future work; design text is not operational scoring enforcement. The current result does not confer regulatory certification, accreditation, independent security assurance, production readiness or permission to publish.

The planned v3 enterprise release is intended to combine the v1.0 three-path foundation with the v2 authorization and integrity controls within a defined enterprise reproduction scope. Full reproduction means reproducing each path's defined outcome, including appropriate HOLD or containment, not forcing all paths into training. V3 remains unreleased and unverified by this v2 exercise. Managed trust, revocation, replay resistance, broader endpoint enforcement and independent assurance require separately scoped work; they are not added to the current evidence merely by appearing in a future-work paragraph.

8.1 Human review in the proposed project profile

The proposed profile separates a technical service from the professional and organizational judgments around it. A technical service can identify materials, preserve provenance, manage review records and enforce the structured permissions actually consumed by its endpoints. A lawyer can assess identified rights questions, intended activities, jurisdictions and unresolved exceptions within an agreed engagement. The responsible organization can then decide whether to authorize a named operation under those conditions.

These roles should not be merged into a single automated approval label. A lawyer's assessment does not automatically issue an operational command. An organizational decision does not change the content of a legal opinion. A successful technical check does not authenticate either person's competence or responsibility. The proposed process records the relationship among the decisions while keeping each within its own scope.

AI assistance may help organize inventories, compare text or prepare questions. Under the proposed project profile, that assistance is not accepted as a substitute for the required qualified human assessment of identified legal questions. This is a stated project policy, not a universal claim that every AI experiment legally requires counsel. The current implementation does not enforce practitioner credentials, and no actual lawyer review is established by the illustrative record.

8.2 The fictional review example

The illustrative record exists to show what a bounded review workflow might need to record. Its status remains `HOLD_PENDING_REAL_REVIEW`, and its organizational disposition remains `NOT_AUTHORIZED_FOR_REAL_OPERATION`. It is not upgraded because the surrounding paper is ready to read or because a synthetic authorization passes technical checks.

A real review would need identified materials and activities rather than an abstract approval of a project name. Relevant questions could include which copy is under review, what use is proposed, which permissions and limitations are relied on, and which exceptions remain unresolved. These are examples of the scope of a review record, not a legal checklist certified as sufficient for every jurisdiction or activity.

The runtime does not consume the fictional practitioner-review JSON as a credential or legal approval artifact. That fact should remain close to any example that looks operational. Otherwise a reader may assume that a signed technical permission implies that the professional-review step has been implemented. The compact papers maintain this distinction explicitly.

8.3 Historical rights and publication statements

Historical publisher statements, embedded notices and proposed later licence drafts are different sources. A release-level statement can clarify what the publisher asserted, but the present manuscript does not infer a universal grant for every third-party expression or later paper. It does not adopt external review drafts merely by listing their hashes.

The 21 July documentary review remains supporting historical context, not practitioner legal clearance or a new frozen candidate. Its proposed canonical-copy and remapping work does not become executed by appearing in a later summary. An unresolved original source is recorded as unresolved rather than replaced by a convenient current file with a similar name.

Final manuscript status is distinct from permission to distribute. The v2 repository and release tag identify this standards edition; v3 remains the planned enterprise release. These identifiers do not themselves establish rights clearance, archive integrity or operational authorization. The external revision handoff identifies editorial files; it is not a legal or publication attestation.

8.4 Limits of the demonstrated technical boundary

Table 13 consolidates the principal limits. The concise reminders next to individual mechanisms remain necessary where removing them would change the meaning of a local claim.

Table 13 Explicit non-claims and the corresponding boundary

Not established	Reason or missing scope
Single-use permission	No consumption counter or replay-prevention state is implemented.
Live revocation	Execution verifies supplied and recorded bindings, not a live authority service.
Professional identity	The technical signature does not verify a practising credential or legal competence.
Complete policy enforcement	Model, environment, step and free-text conditions are not structured authorization checks.
Production key management	Disposable/local keys do not establish custody, rotation or enterprise trust distribution.
Universal filesystem protection	Tests cover specified paths and side effects, not every reparse type or concurrent mutation.
Complete sandboxing	The synthetic import/socket guard is not an operating-system isolation proof.
Model improvement	No new training or common held-out benchmark is reported.
New CSC assurance	Historical qualified scores and a design contract are not a current scorer.
Independent certification	Internal technical verification is not an accredited assurance engagement or regulator approval.

These limits do not make the bounded observations meaningless. They specify the conditions under which the evidence is useful. A standards reviewer can compare the implemented checks, identify what a different deployment would need and avoid treating a narrow demonstration as a universal guarantee. The paper's contribution is strongest when that boundary is visible rather than hidden in a final disclaimer.

8.5 Planned v3 enterprise reproduction

The planned v3 enterprise release would combine the earlier three-path execution foundation with the v2 authorization and integrity controls for a defined enterprise scope. Its inputs, environments and permitted activities would need to be identified independently. Reusing a

historical result or the compact synthetic fixture cannot by itself establish that broader reproduction.

The intended outcome is not to force every path to train. Source-first lifecycle promotion, a runtime package held below its threshold and contained external onboarding can all be correct outcomes under their own contracts. Full reproduction would preserve these distinctions and record the actual positive and negative behavior for the selected source sets.

Any new model-quality claim would require an appropriate evaluation design rather than relying only on a decreasing training loss. Any new operational endpoint would require its own implementation and evidence. Any stronger rights, key-management or assurance claim would need the corresponding responsible process. These are future scope conditions, not additional work authorized by publishing the compact v2 paper pair.

8.6 Separate CSC research

The proposed CSC governance snapshot distinguishes historical records, observation status and future confidence or calibration rules. It is marked `DESIGN_BASELINE_NOT_YET_ENFORCED`. Retaining old and new records does not automatically justify lowering or raising a score; a versioned and validated combination rule would be required.

Future work may investigate direct and proxy evidence, coverage limits, ageing, sector weighting and uncertainty. The present paper does not choose new numeric thresholds or claim that such methods have been calibrated. Historical scores keep their original qualifications and source identities. The CSC Assurance Record direction remains separately scoped rather than a feature delivered by the compact v2 publication or automatically promised for v3.

Conclusion

The reference implementation makes a limited but useful separation explicit: technical eligibility is not permission, signed permission is not package integrity, and neither establishes legal authority or release approval. The reusable transition account identifies the object, evidence, conditions, decision and bounded output so that these distinctions can be inspected rather than inferred.

The September synthetic observations support the named control checks and expected rejection paths. They do not establish new model quality, complete enterprise reproduction, new CSC ratings or independent professional assurance. Historical v1.0 and July candidate observations remain useful comparisons without being treated as current results. TR010 explains the method and reference protocol; TR010-R reports what the verification establishes and where its limits remain. Both papers form the compact v2.0 Standards Edition. The planned v3 enterprise release is separate from the results reported here.

References

Published background

The entries below distinguish paper identifiers from release dates and package locators. A tagged file location is not an independent validation of that paper's datasets. Exact edition details not established by the inspected records are not invented.

[B1] Access-PoD. APOD-TR-002, Access-PoD Governance Framework. Foundational report identified in the original TR010 manuscript. Its exact standalone edition and publication date were not established from the inspected controlled sources. The Access-PoD core release catalogue is background navigation, not a substitute identity for the paper.

[B2] Access-PoD. APOD-TR-006, Version E, Access-Mode method. The publisher's release description identifies Version E as the canonical method paper. The v1.0-access-mode release was published on 5 February 2026; that is the release date, not an independently inferred date printed inside the paper.

[B3] Access-PoD. APOD-TR-006-E.2, informative Working-PoD companion to Version E. The same Access-Mode publication record distinguishes artifact-state interpretation from the method and states that the companion does not extend Version E. A more specific internal edition date is not asserted here.

[B4] APOD-TR-007, Governed Dataset Onboarding and Lifecycle Routing. Tagged paper, distributed with v1.0-qpath-dataset-lifecycle. The supplied reference audit reports the release date as 12 March 2026.

[B5] APOD-TR-008, Interaction-Role Signal Exchange. Paper at the enterprise v1.0 tag. Its historical scope and notices remain distinct from the new paper pair.

[B6] APOD-TR-009, Access PoD CSC Session Model and Execution. Tagged paper.

[B7] APOD-TR-009-R, Access PoD CSC Results. Tagged results paper. B5-B7 are distributed with the enterprise v1.0 release, reported by the supplied reference audit as published on 18 April 2026 and anchored to commit `0fa94ecf47d698a894bc868545f110570d0dec50`.

[B8] QPATH-ARCH-001, Q-Pathformer Multi-state Machine Learning and LLM Training. Paper at the lifecycle tag. The tagged directory confirms the file title and location; no independent internal edition date or architecture benchmark is inferred.

[B9] Access-PoD Artifact Pack v1a. Published public-review pack, published on 28 December 2025. The publisher distinguishes its curated pack from automatically generated source archives and disclaims certification or regulatory approval. This is background for the historical CSC records, not a new certification scheme.

[B10] Access-PoD governed dataset lifecycle release. v1.0-apod-dataset-lifecycle. Its governance artifacts remain distinct from the Q-Pathformer execution tooling.

Current v2 standards paper pair

[P1] Anthony Siljeg. APOD-TR-010. Fail-Closed Governed Dataset Lifecycles and Explicit Training Authorization for AI Training Systems. Compact standards edition, 6 September 2026. Final manuscript for the v2.0 Standards Edition; method and reusable transition protocol.

[P2] Anthony Siljeg. APOD-TR-010-R. Verification Results and v1.0 Comparison. Compact standards edition, 6 September 2026. Final manuscript for the v2.0 Standards Edition; bounded September results, historical comparisons and source catalogue.

The preceding DOCX and reading-copy PDF identities are recorded in `TR010_R02_EDITORIAL_HANDOFF.json`. Earlier handoffs and E01/E20 and edition ledgers retain their dated scope. Final publication PDF identities are recorded separately after conversion and review; neither paper embeds its own final hash in its body. `EVIDENCE_AVAILABILITY_CROSSWALK.json` is the dated crosswalk locator; the two historical ledger companions identify E01-E26 and E27-E28. Their inclusion with this paper pair is not established by these references.

V2.0 Standards Edition release

Repository: `qpathformer/qpathformer-enterprise-v2.0-standards-release`.

Release tag: `v2.0-qpathformer-enterprise-standards`.

V2.0 Standards Edition release.

The repository and release tag identify this v2.0 Standards Edition, separately from the historical July candidate. The compact edition reports bounded synthetic verification, not full enterprise reproduction. These identifiers do not themselves confer rights clearance, independent assurance or operational authorization.

Planned v3 enterprise destination not published

Repository: `qpathformer/qpathformer-enterprise-v3.0-release`.

Proposed tag: `v3.0-qpathformer-enterprise`.

Planned v3 enterprise destination not published.

The proposed enterprise scope combines the v1.0 foundation with v2 controls for defined end-to-end reproduction. It is not a verified v2 result. The Q-Pathformer organization directory and Access-PoD release catalogue provide navigation; they do not establish publication of the planned v3 release.

The 6 September 2026 reference audit records the historical tagged locations and the scope of the checks performed. No release ZIP, historical model output or full dataset was downloaded or re-executed for those reference checks. A release description, tag, commit, file hash and sealed archive digest are different identities. No new licence grant or legal clearance is inferred from a reference link.

Appendix A Implementation and evidence locators

These locators identify the inspected local source selection; local presence does not establish inclusion in the distributed release. The historical handoff describes the earlier editorial source selection; the preceding editorial handoff identifies its own manuscript snapshot. TR010-R Appendix A retains the E01-E28 catalogue and evidence-strength distinctions.

A.1 Current implementation sources

[S1] Authorization representation and verification.
`scripts/training_authorization.py`; `scripts/create_training_authorization_v2.py`; `shared/schemas/authorization/training_authorization_v2.schema.json`. Relevant anchors are `canonical_authorization_bytes`, `validate_training_authorization`, `_validate_scope_contract` and `require_local_lora_scope`. These sources define the current contract; they are not evidence that every possible input was tested.

[S2] Package integrity and key representation. `scripts/package_sealing.py`; `scripts/crypto_keys.py`; `shared/schemas/authorization/package_seal_v2.schema.json`. Relevant anchors are `build_package_inventory`, `_leaf_hash`, `calculate_merkle_root`, `canonical_package_seal_bytes`, `validate_package_seal` and `public_key_sha256`.

[S3] Preparation migration and execution. `scripts/run_training_state1.py`; `scripts/migrate_training_package_authorization_v2.py`; `scripts/run_state1_finetune_local_lora.py`. These are the current local scope-enforcing endpoints. They do not establish production, provider, publication or deployment endpoints.

[S4] Focused unit tests. `tests/unit/test_fail_closed_lifecycle.py`; `tests/unit/test_package_sealing_v2.py`; `tests/unit/test_standards_single_scope.py`; `tests/unit/test_training_authorization_v2.py`. Other retained suite modules concern deterministic export and the private manuscript allowlist. Named executed outcomes are in S6, not inferred merely from the existence of a test function.

[S5] Synthetic harness and local route. `tools/rehearse_standards_release.py`; `tools/standards_rehearsal_child.py`; `docs/execution/TR010_STANDARDS_REFERENCE_RUNBOOK.md`; `requirements-release.txt`. The internal preparation checker is `tools/verify_private_workspace.py`; it is not a generic public two-paper-pack validator.

[S6] September executed records.
`evidence/v2_validation/standards_consolidation_20260905/unit_tests.txt` and `evidence/v2_validation/standards_consolidation_20260905/rehearsal.json`.

[S7] September environment and source identities. `evidence/v2_validation/standards_local_20260905/environment.json`; the adjacent `source_inventory.json`; and `release/provenance/standards_consolidation_20260905/SOURCE_CAPTURE.json`. The recorded parent commit does not alone identify the uncommitted working bytes.

A.2 Historical and documentary sources

[H1] Historical v1.0 comparison. `release/provenance/historical_records/TR010_DRAFTING/QPATHFORMER_V2_TR010_RESULTS_AND_V1_COMPARISON.md`. The source reports original model and onboarding findings; the complete original run sets were not newly reopened.

[H2] Historical M06 export. `release/provenance/historical_records/M06/V2_EXPORT_REHEARSAL_REPORT.json`.

[H3] Corrected historical M10. `release/provenance/historical_records/M10/V2_M10_RERUN_AFTER_M12_REMEDIATION_REPORT.json`, recorded 18 July 2026.

[H4] Corrected historical M11. `release/provenance/historical_records/M11/V2_M11_REBUILD_AFTER_M12_REMEDIATION_REPORT.json` and its retained read-only verification. The report records readiness for M12 reconsideration, not a new release approval.

[H5] Historical human and technical decisions. The M12 records under `release/provenance/historical_records/M12/`, including `HUMAN_RELEASE_REVIEW_20260718.md`, `M12_REMEDIATION_AUTHORIZATION_20260718.md` and `V2_M12_TECHNICAL_REVIEW_REPORT.json`.

[H6] Historical canonical equivalence. `release/provenance/historical_records/M09/V2_M09_CANONICAL_EQUIVALENCE_RESOLUTION_REPORT.json`. Its object-specific rule is not applied to unrelated evidence.

[H7] Historical CSC qualification. The M08 qualification report and `V1_CSC_EVIDENCE_MANIFEST.json` under `release/provenance/historical_records/M08/`. The separate `V2_CSC_GOVERNANCE_SNAPSHOT_CONTRACT.json` is a design baseline, not an active scorer.

[D1] History and review boundaries. `history_notice.md`; `docs/legal-review/ILLUSTRATIVE_LEGAL_REVIEW_WORKED_EXAMPLE.md`; `release/provenance/EXTERNAL_REFERENCES.json`. External legal-review drafts remain not adopted. The fictional review is not a practitioner opinion.

[D2] Preserved source ledgers. `evidence/v2_validation/tr010_results_paper_20260905/source_ledger.json` and `evidence/v2_validation/tr010_results_requirements_20260906/source_ledger.json`. The separate current crosswalk identifies the E01 manuscript revision difference without repairing either original ledger.